

EdgeMind: MCP Without the Cloud

Expert audience Q&A — MCP Dev Summit Mumbai

18

total questions

6

categories

7

hard questions

3

known issues to fix

MCP architecture

- 01 **Is this actually MCP-compliant? The spec defines JSON-RPC 2.0 over stdio/SSE. Your McpServer is just an in-process Kotlin class.**

hard

SHORT ANSWER

No — it is not wire-protocol compliant. EdgeMind implements the MCP conceptual model, not the transport layer.

- 01 **Is this actually MCP-compliant? The spec defines JSON-RPC 2.0 over stdio/SSE. Your McpServer is just an in-process Kotlin class.**

hard

SHORT ANSWER

No — it is not wire-protocol compliant. EdgeMind implements the MCP conceptual model, not the transport layer.

DETAILED EXPLANATION

The MCP spec assumes a network boundary — JSON-RPC 2.0 over stdio or SSE exists to cross that boundary. On a single device with no network surface, that transport is pure overhead. EdgeMind keeps the meaningful abstractions — named tools, scoped capability tokens, a server that enforces guardrails before routing — and replaces the wire format with direct Kotlin coroutine dispatch. If spec compliance were required, wrapping `McpServer.handleToolCall()` in a JSON-RPC 2.0 handler over a local Unix socket is a one-file change.

■ Summit framing: 'We implement MCP semantics, not MCP transport. On a device with no network boundary, the transport layer is irrelevant — what matters is the orchestration contract.'

- 03 **You have one agent hardcoded as 'assistant-agent' with `AgentRole.ELEVATED`. Where is the multi-agent orchestration you are claiming?**

medium

SHORT ANSWER

The infrastructure supports multiple agents — the demo exercises only one. All agents hardcoded to `ELEVATED` is a known bug.

03 You have one agent hardcoded as 'assistant-agent' with AgentRole.ELEVATED. Where is the multi-agent orchestration you are claiming?

medium

SHORT ANSWER

The infrastructure supports multiple agents — the demo exercises only one. All agents hardcoded to ELEVATED is a known bug.

DETAILED EXPLANATION

McpClient and McpServer are designed for multiple clients — ToolRegistry is shared, tokens are per-agent, and AuditJournal tracks by agentId. A real multi-agent scenario instantiates: DEFAULT client (SLM + knowledge read), ELEVATED client (+ LLM access), SYSTEM client (+ audit write). All share the same McpServer. The most impactful 30 minutes before the summit is instantiating a DEFAULT client and showing ToolResult.Unauthorized when it tries to call the LLM tool.

■ Known issue: McpClient.kt:67 hardcodes ELEVATED. Instantiate a second DEFAULT client to demonstrate scope enforcement live.

13 What is the confidence threshold of 0.75 based on? Is it empirically validated or just a chosen constant?

easy

SHORT ANSWER

It is a chosen constant — not empirically validated on a held-out domain-specific dataset.

13 What is the confidence threshold of 0.75 based on? Is it empirically validated or just a chosen constant?

easy

SHORT ANSWER

It is a chosen constant — not empirically validated on a held-out domain-specific dataset.

DETAILED EXPLANATION

The 0.75 threshold controls the cost/latency vs quality tradeoff. Below 0.75 → RAG fallback (+50ms). Still below → LLM escalation (+1-5s). A higher threshold escalates more aggressively to RAG/LLM; lower keeps more in SLM. The right value is domain-dependent — medical triage warrants a higher threshold than general chat. The honest answer: '0.75 is a reasonable starting point — the on-device LoRA training loop will eventually learn domain-specific confidence calibration from accepted interactions, which is a better long-term answer than a fixed constant.'

02 Your `IamService.verifyToken()` says 'in production: verify EC signature' but doesn't actually verify it. So any JWT string passes?

hard

SHORT ANSWER

Yes — signature verification is skipped in this demo. Token issuance is real; verification is payload-only parsing.

02 Your `IamService.verifyToken()` says 'in production: verify EC signature' but doesn't actually verify it. So any JWT string passes?

hard

SHORT ANSWER

Yes — signature verification is skipped in this demo. Token issuance is real; verification is payload-only parsing.

DETAILED EXPLANATION

Token issuance is genuine — Android Keystore generates an EC P-256 keypair and the JWT is signed with SHA256withECDSA inside the hardware TEE. The private key never leaves the secure enclave. However, `verifyToken()` currently only parses the payload. In this demo the issuer and verifier are the same process — a forged token cannot arrive since there is no network surface. In a multi-process or multi-device scenario, the signature check is mandatory.

■ Known issue to fix: add `Signature.getInstance("SHA256withECDSA").verify(signatureBytes)` inside `verifyToken()`. Takes ~20 minutes.

05 The `GuardModel` regex patterns are visible in the APK. An attacker can just avoid those exact strings and bypass the guard entirely.

hard

SHORT ANSWER

Correct for the heuristic layer. The real guard is the 60M-parameter neural classifier which is opaque and cannot be pattern-matched against.

05 The GuardModel regex patterns are visible in the APK. An attacker can just avoid those exact strings and bypass the guard entirely.

hard

SHORT ANSWER

Correct for the heuristic layer. The real guard is the 60M-parameter neural classifier which is opaque and cannot be pattern-matched against.

DETAILED EXPLANATION

Two guard layers: (1) `heuristicClassify()` — regex patterns for obvious cases, fast pre-filter when native is unavailable. (2) `nativeClassify()` — a 60M-parameter .pte neural classifier that runs in 22ms on the HTP. The 22ms latency is the giveaway — a rule-based filter runs in microseconds. The security model assumes the device owner is the adversary's target, not someone with APK access. APK-level attackers have already compromised the device.

■ Known issue: `nativeClassify()` is currently a stub. The real classifier must be wired in before the summit — otherwise the only guard is the visible regex layer.

12 Why does GuardModel run before IAM validation, not after? Seems wasteful to classify prompts that might have an invalid token anyway.

easy

SHORT ANSWER

Safety is more important than efficiency — a blocked prompt should never reach the token check, even on an invalid token path.

12 Why does GuardModel run before IAM validation, not after? Seems wasteful to classify prompts that might have an invalid token anyway.

easy

SHORT ANSWER

Safety is more important than efficiency — a blocked prompt should never reach the token check, even on an invalid token path.

DETAILED EXPLANATION

The ordering (GuardModel → IAM → ToolRegistry → Handler) is deliberate. Running the guard first means zero tokens are ever processed by Phi-3 or the 7B model if GuardModel blocks — even with a valid elevated-scope token. The 22ms guard latency is less than a network round-trip, so the cost is negligible. Blocked prompts never incur the Android Keystore lookup cost — a mild efficiency benefit for high-volume block scenarios.

■ Summit punchline: 'Our safety layer adds less latency than a network round-trip to a cloud guardrails API — and it runs before any LLM token is processed.'

04 **Your benchmark shows 180ms prefill for Phi-3 Mini at 128 tokens. Snapdragon 8 Gen 3 HTP peak is ~10 TOPS. Does the math check out?** hard

SHORT ANSWER

Yes — 180ms is plausible when accounting for memory bandwidth bottlenecks, not just compute throughput.

04 **Your benchmark shows 180ms prefill for Phi-3 Mini at 128 tokens. Snapdragon 8 Gen 3 HTP peak is ~10 TOPS. Does the math check out?** hard

SHORT ANSWER

Yes — 180ms is plausible when accounting for memory bandwidth bottlenecks, not just compute throughput.

DETAILED EXPLANATION

Phi-3 Mini int4 is ~2.1 GB model weight. At 128 tokens, prefill is a matrix multiply over ~3.8B parameters quantised to 4-bit — roughly 1.9 GOPS. At 10 TOPS peak and LPDDR5X at ~77 GB/s, 180ms is realistic. The critical constraint is memory bandwidth, not compute — loading 2.1 GB of weights at 77 GB/s gives ~27ms of pure memory load time alone. QAI Hub handles HTP-specific op lowering and operator fusion.

Decode: 90ms / 64 tokens = ~1.4ms per token → ~714 tok/s decode throughput

■ Key insight: for LLM inference on-device, the bottleneck is almost always memory bandwidth (GB/s), not arithmetic throughput (TOPS). This distinction will come up.

07 **Why use FAISS on Android rather than SQLite FTS5 with vector extension, or an ONNX-based ANN search?** medium

SHORT ANSWER

FAISS gives exact control over index type and fits the 5k-vector demo at ~10MB. SQLite vector extensions are still experimental.

07 **Why use FAISS on Android rather than SQLite FTS5 with vector extension, or an ONNX-based ANN search?** medium

SHORT ANSWER

FAISS gives exact control over index type and fits the 5k-vector demo at ~10MB. SQLite vector extensions are still experimental.

DETAILED EXPLANATION

FAISS IndexFlatL2 over 5k vectors at 384 dimensions fits in ~10MB and returns results in 4ms on XNNPACK. The flat index is exact — no approximation error. For a clinical KB growing to 100k+ chunks, switch to an IVF index for sub-linear search. SQLite vector extensions lack stable arm64-v8a builds. For a larger production system, SQLite + vss would reduce the native dependency surface.

5k vectors × 384 dims × 4 bytes = ~7.7 MB index size

- 11 **The MetricsOverlay shows a randomly fluctuating 50-80% NPU value. How are you actually reading NPU utilisation on Android?**

medium

SHORT ANSWER

Currently it is simulated. Android has no public NPU utilisation API — known issue to fix before the summit.

- 11 **The MetricsOverlay shows a randomly fluctuating 50-80% NPU value. How are you actually reading NPU utilisation on Android?**

medium

SHORT ANSWER

Currently it is simulated. Android has no public NPU utilisation API — known issue to fix before the summit.

DETAILED EXPLANATION

Android has no public API for NPU utilisation. Real NPU utilisation on Snapdragon requires Qualcomm's proprietary QNN profiling API (per-layer execution times) or `/sys/class/kgsl/` nodes (GPU only, not HTP). The fluctuating 50-80% is a random value. Before the summit: remove NPU% from the overlay, relabel it 'simulated', or wire in QNN profiling data from the ExecuTorch backend delegate.

■ Known issue: MetricsOverlay NPU% is randomly generated. Remove or relabel 'simulated' — 5 minute fix.

- 18 **Greedy decoding produces repetitive outputs for longer LLM generations. Why not use beam search or sampling with temperature?**

hard

SHORT ANSWER

Greedy decoding is deliberate for clinical safety — deterministic, reproducible, and auditable. Temperature sampling breaks the audit trail story.

- 18 **Greedy decoding produces repetitive outputs for longer LLM generations. Why not use beam search or sampling with temperature?**

hard

SHORT ANSWER

Greedy decoding is deliberate for clinical safety — deterministic, reproducible, and auditable. Temperature sampling breaks the audit trail story.

DETAILED EXPLANATION

In a regulated medical context, the audit trail logs every inference decision. Temperature sampling means the same prompt produces different outputs on re-runs — breaking the reproducibility guarantee that makes the audit log meaningful. Greedy decoding (argmax at each step) is deterministic: given the same model weights and prompt, you always get the same output — meaning you can reproduce any logged inference from the audit trail. Beam search would improve output quality at 2-5x latency per token but requires careful memory management.

Training & LoRA

- 06 **You call this federated learning. But there is no aggregation server, no secure aggregation, no differential privacy, and no global model improvement. That is just local fine-tuning.**

hard

SHORT ANSWER

Correct. The accurate framing is 'client-side FL' — the device-side part of federated learning, without the aggregation server.

- 06 **You call this federated learning. But there is no aggregation server, no secure aggregation, no differential privacy, and no global model improvement. That is just local fine-tuning.**

hard

SHORT ANSWER

Correct. The accurate framing is 'client-side FL' — the device-side part of federated learning, without the aggregation server.

DETAILED EXPLANATION

True FL (Google FedAvg, PySyft, Flower) requires: aggregation server, secure aggregation, differential privacy, multiple participating devices, and global model improvement. EdgeMind has none of these. What it does match: data stays on device, model updates not raw data, PII protection via `anonymise()`, asynchronous training via `WorkManager`. The LoRA adapter could theoretically be shared as a model delta — that would be the aggregation step — but it never is.

■ Recommended summit framing: 'federated learning without the federation — client-side only, maximum privacy, zero coordination overhead.'

- 10 **LoRA training on 500 samples with rank 8 for 3 epochs — what is the actual memory footprint and how do you stay within 800MB?**

medium

SHORT ANSWER

Layer-by-layer fragmentation — only one transformer layer's activations and gradients are in memory at a time.

10 LoRA training on 500 samples with rank 8 for 3 epochs — what is the actual memory footprint and how do you stay within 800MB?

medium

SHORT ANSWER

Layer-by-layer fragmentation — only one transformer layer's activations and gradients are in memory at a time.

DETAILED EXPLANATION

Phi-3 Mini has 32 transformer layers. Full backprop storing all activations simultaneously is impractical on mobile. LoRA reduces trainable parameters to ~0.1% of total. Layer-by-layer fragmentation: forward pass layer N → store only layer N activations → backward pass layer N → discard → move to N-1. Combined with gradient checkpointing (recompute activations on backward pass instead of storing them), peak memory stays inside ~800MB.

LoRA params: rank=8, alpha=16 → ~4M trainable params vs 3.8B total = 0.1%

14 The training record hardcodes confidence as 0.85f. How does the model know which training samples are high quality vs low quality?

hard

SHORT ANSWER

Currently it does not — all samples are treated equally due to the hardcoded 0.85f. Known bug to fix before the summit.

14 The training record hardcodes confidence as 0.85f. How does the model know which training samples are high quality vs low quality?

hard

SHORT ANSWER

Currently it does not — all samples are treated equally due to the hardcoded 0.85f. Known bug to fix before the summit.

DETAILED EXPLANATION

Without the actual confidence score, the training loop treats a low-confidence LLM response the same as a high-confidence SLM response. The accepted: Boolean flag in InteractionRecord is never set by user action — everything is accepted by default. True quality-weighted training requires both the actual confidence score and user feedback signals (thumbs up/down).

```
// current
dataset.record(prompt, responseText, "slm", 0.85f)

// fix
dataset.record(prompt, responseText, path, parseConfidenceFromResponse(result.data) ?: 0.85f)
```

■ Known bug: AssistantViewModel.kt:181 — replace hardcoded 0.85f with parseConfidenceFromResponse(result.data) ?: 0.85f. 2-minute fix, high impact.

08 What happens if the .pte files are not in assets? Does the app crash on launch?

easy

SHORT ANSWER

No crash — every engine falls back to Kotlin mock implementations. Full demo runs on any Android 10+ device or emulator.

08 What happens if the .pte files are not in assets? Does the app crash on launch?

easy

SHORT ANSWER

No crash — every engine falls back to Kotlin mock implementations. Full demo runs on any Android 10+ device or emulator.

DETAILED EXPLANATION

ExecuTorchRunner catches UnsatisfiedLinkError on `System.loadLibrary('executorch_jni')` and sets `nativeAvailable = false`. Every inference engine checks `runner.isReady()` before native code and falls through to mock implementations. Benchmark numbers on the slides are only valid on Snapdragon 8 Gen 3 HTP hardware.

No .pte files + no native libs → full mock demo on any Android 10+ device

■ Summit tip: always have mock mode ready as a fallback. Demo gods are real.

16 Why does the Vulkan backend exist if HTP handles SLM and LLM inference? When does it actually fire?

easy

SHORT ANSWER

Vulkan is a GPU fallback for the LLM path when HTP headroom is saturated by other apps.

16 Why does the Vulkan backend exist if HTP handles SLM and LLM inference? When does it actually fire?

easy

SHORT ANSWER

Vulkan is a GPU fallback for the LLM path when HTP headroom is saturated by other apps.

DETAILED EXPLANATION

The Snapdragon 8 Gen 3 HTP has limited concurrent capacity. If another app (camera computational photography, on-device speech recognition) locks HTP resources, the LLM path falls back to Vulkan GPU compute. Vulkan is slower than HTP for transformer workloads but faster than XNNPACK CPU for the 7B model. Backend priority: HTP (primary) → Vulkan (GPU fallback) → XNNPACK (CPU last resort). XNNPACK handles RAG embeddings exclusively — MiniLM at 384 dimensions does not benefit from HTP's large-batch optimisation.

Governance

- 09 **The HMAC-signed audit log is on-device in Room. How does it prevent the device owner from tampering with their own audit log?**

medium

SHORT ANSWER

It does not prevent a determined device owner from tampering — HMAC proves entry integrity, not chain custody across a tampered database.

- 09 **The HMAC-signed audit log is on-device in Room. How does it prevent the device owner from tampering with their own audit log?**

medium

SHORT ANSWER

It does not prevent a determined device owner from tampering — HMAC proves entry integrity, not chain custody across a tampered database.

DETAILED EXPLANATION

HMAC-SHA256 proves a specific entry was not modified after creation — change a field and the hash will not match. But a rooted device owner could delete entries from Room entirely. The threat model addresses: (1) proving to a regulator what happened given a cooperative device, and (2) detecting accidental or app-level tampering. For stronger guarantees: remote attestation (Play Integrity), log sync to a write-only cloud endpoint, or a hardware-backed append-only log.

■ Honest framing: 'The architecture is GDPR-compatible by design. Production compliance for adversarial scenarios requires remote attestation on top.'

- 15 **You claim GDPR and DPDP compliance. What specific articles does the design address and what is still missing?**

medium

SHORT ANSWER

The architecture addresses data minimisation, storage limitation, integrity, and accountability — but misses erasure, access rights, and consent.

15 You claim GDPR and DPDP compliance. What specific articles does the design address and what is still missing?

medium

SHORT ANSWER

The architecture addresses data minimisation, storage limitation, integrity, and accountability — but misses erasure, access rights, and consent.

DETAILED EXPLANATION

Addressed: data minimisation (Art. 5) via PII anonymisation; storage limitation — gradients never leave device; integrity via HMAC-signed audit log and Android Keystore; accountability via full agent action logging. Missing: right to erasure (Art. 17) — no mechanism to delete interaction records; right to access (Art. 15) — no portable data export; consent management — no explicit consent flow before training on interactions; Data Protection Impact Assessment not demonstrated.

■ Honest framing: 'The architecture is GDPR-compatible by design — data never leaves the device. Production compliance requires erasure, access, and consent flows on top.'

17 The Aadhaar pattern in GuardModel has false positive risk. What specific pattern are you using and why is it risky?

easy

SHORT ANSWER

A 12-digit number regex will false-positive on phone numbers, order IDs, and timestamps — all common in clinical contexts.

17 The Aadhaar pattern in GuardModel has false positive risk. What specific pattern are you using and why is it risky?

easy

SHORT ANSWER

A 12-digit number regex will false-positive on phone numbers, order IDs, and timestamps — all common in clinical contexts.

DETAILED EXPLANATION

Aadhaar numbers are 12 digits (XXXX XXXX XXXX). A naive regex like `\d{12}` also matches: 12-digit phone numbers (common in India), medical record numbers, timestamps in milliseconds, long order IDs. In a clinical context the false positive rate could be high enough to block legitimate queries. The fix: add Verhoeff checksum validation (Aadhaar uses it), prefix exclusion (valid Aadhaar starts with 2-9), and context windowing.

■ Known issue: add Verhoeff checksum validation to reduce Aadhaar false positives in clinical text.